

Тестове завдання: Senior Frontend Engineer

Smart Sender

Загальне

Реалізуйте невеликий застосунок: вхід, список вебхуків і редагування.

Виділіть на завдання не більше 4 годин. Термін здачі — 7 днів. Якщо щось залишиться незавершеним, зазначте це в README. Оцінюємо якість реалізованого, технічні рішення та пріоритети.

Дизайн довільний, можна використовувати будь-яку UI-бібліотеку. Оцінюємо логіку, архітектуру та читабельність коду, а не візуальне оформлення.

Стек

- React + TypeScript у strict mode.
- MSW для імітації API — реалізуйте самостійно за контрактом нижче.
- Решта бібліотек — на ваш вибір.

Що реалізувати

1. Вхід і сесія

- Форма входу: email і пароль. Тестові облікові дані вкажіть у README.
- Після входу — обмін `device_session_token` на сесію (`/auth/token/issue`), завантаження `/v1/me` і перехід до списку вебхуків.
- `device_session_token` зберігається лише в пам'яті — не в `localStorage` і не в URL.
- `fingerprint` — стабільний ідентифікатор пристрою (32 hex-символи), генерується один раз, зберігається в `localStorage` і передається в auth-запитах відповідно до контракту.
- Токени сесії клієнт не бачить: сесію зберігає мок (аналог `HttpOnly-cookie`).
- Сесія діє 30 секунд, після цього захищені запити повертають `401`. Клієнт викликає `/auth/token/rotate` і повторює запит один раз.
- Якщо кілька запитів одночасно отримали `401`, вони використовують **один спільний rotate**. Сам rotate не запускає новий rotate.
- Помилка rotate або повторний `401` — завершення локальної сесії та повернення на екран входу.
- Вихід: `/auth/token/revoke`, очищення локального стану та кешу користувацьких даних.

2. CSRF

- Усі запити надсилаються із заголовком `X-Requested-With: XMLHttpRequest`.
- Перед першими іншими API-запитами виконайте `GET /csrf`. Токен із заголовка `X-CSRF-TOKEN` передавайте в однойменному заголовку на `POST` і `PUT`.
- Буде перевагою: на `419` повторно отримати CSRF-токен і повторити запит не більше одного разу.

3. Список вебхуків

- Таблиця: назва, URL, активність.
- Пагінація по 10 записів і пошук за назвою.
- Сторінка та пошук зберігаються в URL і відновлюються при перезавантаженні та навігації «назад» / «вперед».
- При зміні пошуку — перехід на першу сторінку.

- Стани завантаження, порожнього результату та помилки.

4. Редагування

- Форма на сторінці або в модальному вікні: назва, URL.
- Серверні помилки валідації відображаються біля відповідних полів.

Контракт API

```
GET /csrf
  → 204, header X-CSRF-TOKEN

POST /auth/login      { email, password, fingerprint }
  → 200 { device_session_token } | 422
POST /auth/token/issue { device_session_token, fingerprint }
  → 200 (starts session) | 422
POST /auth/token/rotate { fingerprint }
  → 200 | 400
POST /auth/token/revoke { fingerprint }
  → 204
GET /v1/me
  → 200 { id, email, first_name, last_name, name } | 401

GET /v1/webhooks?page=&limit=&search=
  → 200 WebhookList | 401
GET /v1/webhooks/{id}
  → 200 Webhook | 401 | 404
PUT /v1/webhooks/{id} { name, url }
  → 200 Webhook | 401 | 422

Webhook      = { id, name, url, active, created_at }
WebhookList = { data: Webhook[],
                paging: { pages: { current, last }, results: { total, limitation } } }
```

Правила мока

- Один користувач із фіксованими email і паролем; 25–30 вебхуків; стан лише в пам'яті.
- `/auth/login` вимагає заголовок `X-Captcha-Token` — мок приймає будь-яке непорожнє значення, віджет капчі не потрібен.
- Невірний пароль — `422` з помилкою поля.
- Сесія діє 30 секунд; `rotate` продовжує її ще на 30 секунд. До успішного `/auth/token/issue` та після `/auth/token/revoke` `rotate` повертає `400`.
- CSRF-токен фіксований. Мок перевіряє його на `POST` і `PUT` до виконання операції; відсутній або некоректний токен — `419`.
- `page` починається з 1; `limit=10`; пошук — підрядок у назві без урахування регістру.
- `results.total` — кількість записів після застосування пошуку, `results.limitation` — розмір сторінки. Порядок записів стабільний.
- Назва обов'язкова, URL — коректна HTTP/HTTPS-адреса.
- Після перезавантаження сторінки допустимий повторний вхід; параметри списку з URL мають застосуватися.

Формат помилки

```
{
  "error": {
    "type": "ValidationException",
    "message": "The given data was invalid.",
    "payload": { "url": ["The url must be a valid URL."] }
  }
}
```

Статус	type	payload
400	<code>BadRequestException</code>	—
401	<code>AuthenticationException</code>	—
404	<code>NotFoundException</code>	—
419	<code>TokenMismatchException</code>	—
422	<code>ValidationException</code>	помилки за полями

Тест і задача

Один автоматизований тест: два паралельні запити з 401 → один rotate → успішні повтори.

Надішліть посилання на GitHub або GitLab. У README вкажіть запуск застосунку й тесту, тестові облікові дані та коротко поясніть ключові рішення.

Що оцінюємо

- Коректність сесії та повторів запитів.
- Розділення API-логіки, стану та UI.
- Роботу з URL, формами та помилками.
- Якість типізації та читабельність коду.

Архітектура має відповідати масштабу завдання: монорепозиторій і складна інфраструктура не потрібні.